# Aprendizaje por Refuerzo *mapa conceptual*

Para visualizar el contenido, se debe hacer click en un Cuadro.

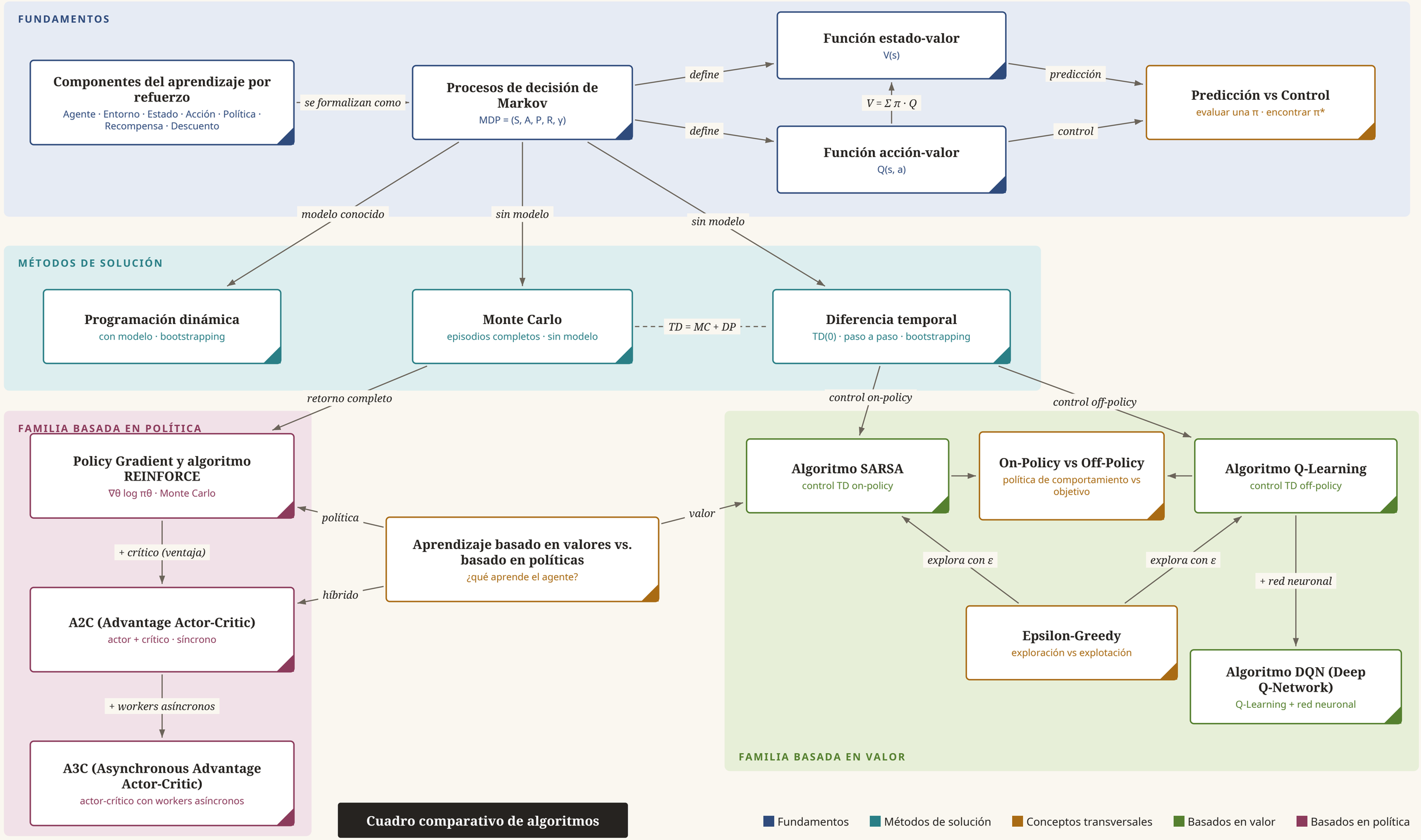

**UNIVERSIDAD GRAN ROSARIO · APRENDIZAJE POR REFUERZO**

**INTEGRANTES**

Acosta Gabriel Hernan Miguel
Benitez Dante Flavian
Mazza Agustin
Rios Facundo
Ruiz Diaz Alejandro

**PROFESORES**

Guadalupe Pascal
Julián E. Tornillo
Cristian Oyarzo


Tocá un cuadro para abrir su página; en cada página, «Volver al mapa» regresa acá.

# Componentes del aprendizaje por refuerzo

*Los ingredientes de cualquier problema de RL*

← Volver al mapa

El **aprendizaje por refuerzo (RL)** es un paradigma en el que un **agente** aprende a decidir **interactuando con un entorno**, por prueba y error, para maximizar la **recompensa acumulada** a largo plazo. No hay respuestas correctas etiquetadas (como en el aprendizaje supervisado): solo una señal de recompensa que dice qué tan bien se está actuando.

### IDEAS CLAVE

- **Ciclo de interacción:** en cada paso t el agente observa el estado $S_t$, elige una acción $A_t$, y el entorno responde con una recompensa $R_{t+1}$ y un nuevo estado $S_{t+1}$.
- **Objetivo:** no maximizar la recompensa inmediata, sino el **retorno** $G_t$ (suma de recompensas futuras descontadas).
- **Dilema exploración–explotación:** probar acciones nuevas (explorar) vs. usar la mejor conocida (explotar).
- **Episodio:** secuencia completa de interacción desde un estado inicial hasta uno terminal. Las tareas pueden ser *episódicas* o *continuas*.

### LOS SIETE COMPONENTES

| Componente | Qué es | Notación |
|---|---|---|
| **Agente** | Quien aprende y toma decisiones (el «cerebro»). | — |
| **Entorno** | Todo lo externo al agente; responde a cada acción con un nuevo estado y una recompensa. | — |
| **Estado** | Descripción de la situación del entorno en un instante dado. | $s \in S$ |
| **Acción** | Decisión que el agente puede tomar en un estado. | $a \in A(s)$ |
| **Política** | Regla de comportamiento del agente: qué acción elegir en cada estado. Es lo que se quiere aprender. | $\pi(a \mid s)$ |
| **Recompensa** | Señal escalar inmediata que evalúa la acción tomada; define el objetivo. | $r = R(s,a,s')$ |
| **Descuento** | Factor que pondera el futuro: $\gamma \approx 0$ → agente «miope»; $\gamma \approx 1$ → visión a largo plazo. | $\gamma \in [0,1]$ |

### FÓRMULAS

*Retorno descontado*

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \Sigma_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

*Política estocástica*

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

### EJEMPLO

***Robot aspiradora.*** *Agente: el robot. Entorno: la casa. Estado: posición + nivel de batería. Acciones: avanzar, girar, volver a la base. Recompensa: +1 por superficie limpiada, −10 por chocar. $\gamma$ = 0,95 para que valore llegar a cargarse antes de quedarse sin batería.*

### RELACIONADO · CLIC PARA ABRIR

Procesos de decisión de Markov — se formalizan como un MDP
Función estado-valor — el retorno define el valor de un estado
Epsilon-Greedy — exploración vs. explotación

# Procesos de decisión de Markov (MDP)

*El marco matemático del aprendizaje por refuerzo*

← Volver al mapa

Un **MDP** es el modelo matemático formal de un problema de decisión secuencial: describe estados, acciones, cómo evoluciona el entorno y qué recompensas se reciben. Casi todo el RL se plantea como la búsqueda de una buena política en un MDP.

### IDEAS CLAVE

- **Tupla (S, A, P, R, γ):** conjunto de estados, de acciones, dinámica de transición P(s'|s,a), función de recompensa R(s,a,s') y factor de descuento γ.
- **Propiedad de Markov:** el futuro depende solo del estado y la acción actuales, no de toda la historia. El estado resume todo lo relevante del pasado.
- **Objetivo:** hallar la política óptima π* que maximiza el retorno esperado desde cualquier estado.
- **Con modelo vs. sin modelo:** si P y R se conocen, el MDP se resuelve por *programación dinámica*; si no se conocen, el agente aprende de la experiencia (*Monte Carlo, Diferencia Temporal*).
- **Variantes:** MDP parcialmente observable (POMDP), donde el agente no ve el estado completo.

### FÓRMULAS

*Propiedad de Markov*

$$P(S_{t+1} \mid S_t, A_t) = P(S_{t+1} \mid S_1, A_1, \ldots, S_t, A_t)$$

*Ecuación de Bellman (esperanza)*

$$V^{\pi}(s) = \Sigma_a \, \pi(a|s) \, \Sigma_{s'} \, P(s'|s,a) \, [\, R + \gamma \, V^{\pi}(s') \,]$$

*Ecuación de Bellman (optimalidad)*

$$V^{*}(s) = \max_a \, \Sigma_{s'} \, P(s'|s,a) \, [\, R + \gamma \, V^{*}(s') \,]$$

### EJEMPLO

***Gridworld.*** *Una cuadrícula 4×4: cada celda es un estado, las acciones son ↑ ↓ ← →, cada paso cuesta −1 y llegar a la meta termina el episodio. Si el piso es resbaloso, P(s'|s,a) es estocástica: ir «↑» a veces te desvía a un costado.*

### RELACIONADO · CLIC PARA ABRIR

Componentes del aprendizaje por refuerzo — formaliza sus componentes
Función estado-valor — define la función estado-valor
Función acción-valor — define la función acción-valor
Programación dinámica — se resuelve con el modelo conocido
Monte Carlo — o aprendiendo de episodios
Diferencia temporal — o aprendiendo paso a paso

# Función estado-valor V(s)

*¿Qué tan bueno es estar en este estado?*

← Volver al mapa

La **función estado-valor** $V^{\pi}(s)$ es el **retorno esperado** si el agente parte del estado s y sigue la política π de ahí en adelante. Resume el «futuro prometedor» de un estado en un solo número.

### IDEAS CLAVE

- Depende de la política: una misma situación vale distinto según cómo se actúe después.
- **Recursividad de Bellman:** el valor de un estado = recompensa inmediata + valor descontado del siguiente estado.
- **Valor óptimo V*(s):** el mayor valor alcanzable por cualquier política; de él se deduce π*.
- **Relación con Q:** V es el promedio de Q ponderado por la política.
- **Limitación para controlar:** con solo V, para elegir acción hace falta conocer el modelo (a qué estado lleva cada acción). Por eso, en control sin modelo se prefiere Q(s,a).
- **Cómo se estima:** programación dinámica (con modelo), promedio de retornos en Monte Carlo, o TD(0).

### FÓRMULAS

*Definición*

$$V^{\pi}(s) = E_{\pi}[\, G_t \mid S_t = s \,]$$

*Bellman*

$$V^{\pi}(s) = E_{\pi}[\, R_{t+1} + \gamma\, V^{\pi}(S_{t+1}) \mid S_t = s \,]$$

*Desde Q*

$$V^{\pi}(s) = \Sigma_a\, \pi(a \mid s) \cdot Q^{\pi}(s,a)$$

*Actualización TD(0)*

$$V(s) \leftarrow V(s) + \alpha\, [\, r + \gamma\, V(s') - V(s) \,]$$

### EJEMPLO

***Gridworld.*** *Con recompensa −1 por paso, una celda adyacente a la meta tiene V ≈ −1 y una celda lejana V ≈ −14: el valor «mide» cuántos pasos faltan. El agente aprende que moverse hacia celdas de mayor V es bueno.*

### RELACIONADO · CLIC PARA ABRIR

Función acción-valor — su versión que incluye la acción
Procesos de decisión de Markov — se define sobre un MDP
Predicción vs Control — estimarla es el problema de predicción
Diferencia temporal — se estima con TD(0)
A2C (Advantage Actor-Critic) — es el «crítico» en actor-crítico

# Función acción-valor Q(s, a)

*¿Qué tan buena es esta acción en este estado?*

← Volver al mapa

La **función acción-valor** $Q^{\pi}(s,a)$ es el retorno esperado si el agente, estando en s, **toma la acción a** y luego sigue la política π. Es la función central del aprendizaje basado en valores.

### IDEAS CLAVE

- **Permite decidir sin modelo:** la política greedy es simplemente $\pi(s) = \text{argmax}_a\ Q(s,a)$.
- **Q*(s,a)** es el valor óptimo; conocer Q* equivale a conocer cómo actuar de forma óptima.
- **Forma tabular:** una «Q-table» con una fila por estado y una columna por acción. Cuando los estados son demasiados, se aproxima con una red neuronal (DQN).
- **Función de ventaja:** $A(s,a) = Q(s,a) - V(s)$ mide cuánto mejor es una acción que el promedio en ese estado (es la que usan A2C/A3C).
- **Es lo que estiman** SARSA ($Q^{\pi}$) y Q-Learning (Q*).

### FÓRMULAS

*Definición*

$$Q^{\pi}(s,a) = E_{\pi}[\, G_t \mid S_t=s,\, A_t=a \,]$$

*Óptima (Bellman)*

$$Q^*(s,a) = \Sigma_{s'}\, P(s'|s,a)\, [\, R + \gamma \max_{a'} Q^*(s',a') \,]$$

*Política greedy*

$$\pi(s) = \text{argmax}_a\ Q(s,a)$$

*Ventaja*

$$A(s,a) = Q(s,a) - V(s)$$

### EJEMPLO

***Q-table.*** *En una celda con meta a la derecha: $Q(s,\rightarrow) = -2$, $Q(s,\uparrow) = -5$, $Q(s,\leftarrow) = -9$, $Q(s,\downarrow) = -6$. La política greedy elige «→». Nunca hizo falta saber a dónde lleva cada acción: basta comparar valores.*

### RELACIONADO · CLIC PARA ABRIR

Función estado-valor — V es el promedio de Q según π
Algoritmo SARSA — la estima on-policy
Algoritmo Q-Learning — la estima off-policy
Algoritmo DQN (Deep Q-Network) — la aproxima con una red neuronal
Epsilon-Greedy — se usa para elegir acciones a partir de Q

# Predicción vs Control

*Dos preguntas distintas sobre una política*

← Volver al mapa

Los problemas de RL se separan en dos tareas: la **predicción** (o evaluación de política) estima **qué tan buena es** una política dada; el **control** busca **la mejor política**. Casi todos los algoritmos de control combinan ambas.

### IDEAS CLAVE

- **Predicción:** dada una política fija π, estimar $V^{\pi}$ o $Q^{\pi}$. No se cambia el comportamiento, solo se mide.
- **Control:** encontrar π* (y V*, Q*). Se logra alternando *evaluar* y *mejorar* la política.
- **Iteración generalizada de la política (GPI):** evaluación y mejora conviven: la evaluación acerca la función de valor a la política actual y la mejora vuelve la política greedy respecto del valor.
- **Control necesita explorar** (por ejemplo con ε-greedy); la predicción pura no.

### PREDICCIÓN VS CONTROL

| | Predicción | Control |
|---|---|---|
| **Pregunta** | ¿Qué tan buena es esta política? | ¿Cuál es la mejor política? |
| **Entrada** | Una política π fija | Ninguna (se parte de una política cualquiera) |
| **Salida** | $V^{\pi}(s)$ o $Q^{\pi}(s,a)$ | π*, V*(s), Q*(s,a) |
| **Ejemplos** | Evaluación de política (DP), MC de predicción, TD(0) | Iteración de políticas/valores (DP), MC control, SARSA, Q-Learning |

### FÓRMULAS

*Predicción (TD)*

$$V(s) \leftarrow V(s) + \alpha\,[\, r + \gamma\, V(s') - V(s)\,]$$

*Control (Q-Learning)*

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$$

### EJEMPLO

***Dos preguntas en un videojuego.*** *Predicción: «si siempre juego de forma defensiva, ¿qué puntaje esperado tengo?». Control: «¿qué estrategia me da el mayor puntaje posible?».*

### RELACIONADO · CLIC PARA ABRIR

Función estado-valor — predicción estima V
Función acción-valor — control suele estimar Q
Programación dinámica — resuelve ambos con modelo
Monte Carlo — resuelve ambos con episodios
Diferencia temporal — resuelve ambos paso a paso

# Programación dinámica (DP)

*Planificar cuando el modelo del entorno es conocido*

← Volver al mapa

La **programación dinámica** resuelve MDPs **cuando se conoce el modelo completo** (P y R). Convierte las ecuaciones de Bellman en reglas de actualización que se aplican iterativamente hasta converger. Es la base teórica de todo el RL.

### IDEAS CLAVE

- **Bootstrapping:** actualiza una estimación a partir de otras estimaciones (el valor de s se calcula con el valor de s').
- **Barridos completos:** recorre *todos* los estados y todas las transiciones posibles (sin muestrear).
- **Evaluación de política:** calcula $V^{\pi}$ aplicando Bellman de esperanza hasta converger.
- **Mejora de política:** hace la política greedy respecto del valor actual.
- **Iteración de políticas:** alterna evaluación completa y mejora. **Iteración de valores:** fusiona ambos pasos con la ecuación de optimalidad.
- **Limitaciones:** requiere un modelo (casi nunca disponible) y su costo crece con el número de estados (maldición de la dimensionalidad).

### FÓRMULAS

*Evaluación de política*

$$V_{k+1}(s) = \Sigma_a \, \pi(a \mid s) \, \Sigma_{s'} \, P(s' \mid s,a) \, [\, R + \gamma \, V_k(s') \,]$$

*Iteración de valores*

$$V_{k+1}(s) = \max_a \, \Sigma_{s'} \, P(s' \mid s,a) \, [\, R + \gamma \, V_k(s') \,]$$

### EJEMPLO

***Planificar una ruta con mapa completo.*** *Si conocés las probabilidades de cada transición, podés calcular V* para todas las celdas sin ni siquiera «jugar» una vez: solo iterando la fórmula hasta que los valores dejan de cambiar.*

### RELACIONADO · CLIC PARA ABRIR

Procesos de decisión de Markov — requiere el MDP completo
Monte Carlo — alternativa sin modelo
Diferencia temporal — TD hereda su bootstrapping
Predicción vs Control — evaluación = predicción; iteración = control

# Métodos Monte Carlo (MC)

*Aprender promediando retornos de episodios completos*

← Volver al mapa

Los métodos **Monte Carlo** aprenden **directamente de la experiencia** sin necesitar el modelo. Esperan a que el episodio termine, calculan el retorno real $G_t$ de cada estado visitado y estiman el valor como el **promedio de esos retornos**.

### IDEAS CLAVE

- **Sin bootstrapping:** usa retornos reales completos, no estimaciones de otros estados.
- **Solo tareas episódicas:** necesita que el episodio termine para poder actualizar.
- **Sin sesgo, alta varianza:** el retorno real es una estimación insesgada, pero varía mucho entre episodios.
- **First-visit vs. every-visit:** promediar el retorno solo de la primera visita al estado en el episodio, o de todas.
- **MC control:** estima Q y mejora la política; necesita explorar (exploring starts o ε-greedy).
- **Ventaja:** puede aprender de simulaciones o experiencia real, y no se ve afectado por violaciones de Markov.

### FÓRMULAS

*Estimación por promedio*

$$V(s) \approx ( G^{(1)} + G^{(2)} + \ldots + G^{(N)} ) / N$$

*Actualización incremental*

$$V(S_t) \leftarrow V(S_t) + \alpha [ G_t - V(S_t) ]$$

### EJEMPLO

***Blackjack.*** *Se juegan muchas manos; al terminar cada una se conoce el resultado final (+1, 0, −1) y se actualiza el valor de todos los estados por los que pasó esa mano. No hace falta conocer las probabilidades del mazo.*

### RELACIONADO · CLIC PARA ABRIR

Diferencia temporal — TD actualiza sin esperar al final
Programación dinámica — DP necesita modelo; MC no
Policy Gradient y algoritmo REINFORCE — REINFORCE usa retornos Monte Carlo
Predicción vs Control — sirve para predicción y control

# Aprendizaje por diferencia temporal (TD)

*Aprender paso a paso, sin esperar al final del episodio*

← Volver al mapa

El aprendizaje por **diferencia temporal** combina lo mejor de Monte Carlo (aprende de la experiencia, sin modelo) y de la programación dinámica (hace bootstrapping). Actualiza el valor de un estado **después de cada paso**, usando la estimación del siguiente estado.

### IDEAS CLAVE

- **Objetivo TD:** r + γ·V(s'). En vez del retorno completo $G_t$, usa la recompensa inmediata más el valor estimado del siguiente estado.
- **Error TD (δ):** diferencia entre el objetivo y la estimación actual; es la señal de aprendizaje.
- **Funciona en tareas continuas** y en episodios inconclusos.
- **Menos varianza que MC** pero con algo de sesgo (porque se apoya en estimaciones).
- **Base de los grandes algoritmos:** SARSA, Q-Learning, DQN y el «crítico» de A2C/A3C.
- **Extensiones:** n-step TD y TD(λ) interpolan entre TD(0) y Monte Carlo.

### COMPARACIÓN DP · MC · TD

| | Prog. dinámica | Monte Carlo | Diferencia temporal |
|---|---|---|---|
| **¿Necesita modelo?** | Sí | No | No |
| **¿Bootstrapping?** | Sí | No | Sí |
| **¿Muestrea?** | No (barrido completo) | Sí | Sí |
| **Actualiza…** | Cada iteración | Al final del episodio | Cada paso |
| **Sesgo / Varianza** | — / — | Sin sesgo / alta varianza | Algo de sesgo / baja varianza |

### FÓRMULAS

*TD(0)*

$$V(S_t) \leftarrow V(S_t) + \alpha\,[\,R_{t+1} + \gamma\,V(S_{t+1}) - V(S_t)\,]$$

*Error TD*

$$\delta_t = R_{t+1} + \gamma\,V(S_{t+1}) - V(S_t)$$

### EJEMPLO

***Volver a casa en auto.*** *Estimás 30 min de viaje. Tras 5 min topás con tráfico y la nueva estimación sube a 40 min: TD corrige ya mismo la estimación inicial. MC esperaría a llegar a casa para ajustar.*

### RELACIONADO · CLIC PARA ABRIR

Monte Carlo — comparte el muestreo, no el bootstrapping
Programación dinámica — comparte el bootstrapping, no el modelo
Algoritmo SARSA — control on-policy basado en TD
Algoritmo Q-Learning — control off-policy basado en TD
A2C (Advantage Actor-Critic) — el crítico se aprende con TD

# Basado en valores vs. basado en políticas

*Dos formas de llegar a una buena política*

← Volver al mapa

Hay dos grandes familias: los métodos **basados en valores** aprenden una función de valor (V o Q) y **derivan** la política de ella; los métodos **basados en políticas optimizan directamente** la política. Los **actor-crítico** combinan ambos.

### IDEAS CLAVE

- **Basados en valores:** la política es implícita (greedy o ε-greedy sobre Q). Ejemplos: SARSA, Q-Learning, DQN.
- **Basados en políticas:** se parametriza $\pi_\theta(a|s)$ (p. ej. una red con softmax) y se ajusta θ por ascenso del gradiente. Ejemplos: REINFORCE.
- **Actor-crítico (híbrido):** el *actor* es la política y el *crítico* es la función de valor que lo guía. Ejemplos: A2C, A3C.

### COMPARACIÓN

| | Basado en valores | Basado en políticas |
|---|---|---|
| **Qué aprende** | V(s) o Q(s,a); la política se deriva | La política $\pi_\theta(a\|s)$ directamente |
| **Acciones** | Discretas (necesita un max sobre acciones) | Discretas y **continuas** |
| **Política** | Determinista (con ε para explorar) | Estocástica de forma natural |
| **Convergencia** | Puede oscilar o divergir con redes | Más suave, pero a óptimos locales |
| **Varianza** | Baja | Alta (se reduce con crítico/baseline) |
| **Eficiencia de datos** | Mayor (off-policy, replay) | Menor (suele ser on-policy) |

### FÓRMULAS

*Valor → política*

$$\pi(s) = \operatorname{argmax}_a Q(s,a)$$

*Política directa*

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

### EJEMPLO

***Conducir un auto.*** *Basado en valor: calcular cuán bueno es cada giro posible y elegir el mejor. Basado en política: la red entrega directamente «girar el volante 3,2°»: no se puede hacer un argmax sobre infinitos ángulos.*

### RELACIONADO · CLIC PARA ABRIR

Algoritmo SARSA — valor
Algoritmo Q-Learning — valor
Algoritmo DQN (Deep Q-Network) — valor profundo
Policy Gradient y algoritmo REINFORCE — política
A2C (Advantage Actor-Critic) — híbrido actor-crítico
A3C (Asynchronous Advantage Actor-Critic) — híbrido asíncrono

# On-Policy vs Off-Policy

*¿La política que explora es la misma que se aprende?*

← Volver al mapa

Se distinguen dos políticas: la de **comportamiento** b (la que genera los datos) y la **objetivo** π (la que se evalúa y mejora). En **on-policy** son la misma; en **off-policy** son distintas.

### IDEAS CLAVE

- **On-policy:** aprende sobre la política que está ejecutando, exploración incluida. Es más simple y estable, pero descarta los datos viejos.
- **Off-policy:** puede aprender la política óptima (greedy) mientras explora con otra, **reutilizar experiencia pasada** (experience replay) o aprender de datos de otros agentes o de humanos.
- **Corrección:** cuando b ≠ π en métodos de muestreo (p. ej. Monte Carlo) se usa *importance sampling*, que reescala por el cociente π/b.
- **Costo:** off-policy + bootstrapping + aproximación de funciones puede ser inestable (la «tríada mortal»).

### CLASIFICACIÓN DE LOS ALGORITMOS

| Algoritmo | Tipo | Por qué |
|---|---|---|
| **SARSA** | On-policy | Su objetivo usa la acción a' realmente elegida por la política ε-greedy. |
| **Q-Learning** | Off-policy | Su objetivo usa max Q(s',·), sin importar qué acción se tome luego. |
| **DQN** | Off-policy | Hereda de Q-Learning y reutiliza datos del replay buffer. |
| **REINFORCE** | On-policy | Aprende con trayectorias generadas por la política actual. |
| **A2C / A3C** | On-policy | Los gradientes se calculan con datos de la política vigente. |

### FÓRMULAS

*Importance sampling*

$$\rho_t = \pi(A_t \mid S_t) \,/\, b(A_t \mid S_t)$$

### EJEMPLO

***Aprender a cocinar.*** *On-policy: aprendés solo de lo que vos mismo cocinás con tu receta actual. Off-policy: aprendés de recetas de otros y de tus intentos viejos, pero apuntando a cocinar la mejor versión posible.*

### RELACIONADO · CLIC PARA ABRIR

Algoritmo SARSA — ejemplo on-policy
Algoritmo Q-Learning — ejemplo off-policy
Algoritmo DQN (Deep Q-Network) — off-policy con replay
Epsilon-Greedy — política de comportamiento típica

# Algoritmo SARSA

*State – Action – Reward – State – Action*

← Volver al mapa

**SARSA** es un algoritmo de **control TD on-policy**. Aprende $Q^{\pi}(s,a)$ usando la quíntupla (s, a, r, s', a') y su objetivo incluye la acción a' **realmente elegida** por la política (ε-greedy) en el siguiente estado.

### IDEAS CLAVE

- **Es on-policy:** evalúa y mejora la misma política que usa para explorar; por eso «paga» por sus propios errores de exploración.
- **Conducta cautelosa:** en Cliff Walking, SARSA aprende un camino seguro, alejado del acantilado, porque sabe que a veces explorará.
- **Convergencia:** converge a π* si todos los pares (s,a) se visitan infinitamente y ε → 0 gradualmente (condición GLIE).

### PASO A PASO

1. Inicializar Q(s,a) arbitrariamente.
2. En el estado s elegir a con política ε-greedy sobre Q.
3. Ejecutar a; observar r y s'.
4. Elegir a' en s' con la **misma** política ε-greedy.
5. Actualizar Q(s,a) con el objetivo r + γ Q(s',a').
6. s ← s', a ← a'. Repetir hasta fin del episodio.

### FÓRMULAS

*Actualización*

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma\, Q(s',a') - Q(s,a)\,]$$

*Objetivo TD*

$$r + \gamma\, Q(s',a') \quad (a' \sim \varepsilon\text{-greedy})$$

### EJEMPLO

***Cliff Walking.*** *Una cuadrícula con un acantilado junto al camino más corto. Con ε = 0,1, SARSA aprende la ruta más larga y segura, porque contempla que una acción aleatoria cerca del borde podría hacerlo caer.*

### RELACIONADO · CLIC PARA ABRIR

Diferencia temporal — es TD aplicado a control
On-Policy vs Off-Policy — ejemplo on-policy
Epsilon-Greedy — elige acciones con ε-greedy
Algoritmo Q-Learning — su contraparte off-policy
Función acción-valor — estima $Q^{\pi}$

# Algoritmo Q-Learning

*Aprender la política óptima mientras se explora*

← Volver al mapa

**Q-Learning** (Watkins, 1989) es un algoritmo de **control TD off-policy**. Su objetivo usa el **máximo** valor de Q en el siguiente estado, es decir, asume que a partir de ahí se actuará de forma óptima, **aunque el comportamiento real sea otro**.

### IDEAS CLAVE

- **Es off-policy:** la política de comportamiento (ε-greedy) explora; la política objetivo (greedy) es la que se aprende.
- **Aprende Q* directamente** sin importar cómo se recolecten los datos, siempre que todos los pares (s,a) se visiten con frecuencia.
- **Más audaz:** en Cliff Walking aprende el camino óptimo pegado al acantilado, aunque durante el entrenamiento cae a veces.
- **Limitaciones:** la Q-table no escala a espacios grandes (→ DQN) y el operador max produce *sobreestimación* (→ Double Q-Learning).

### SARSA VS Q-LEARNING

| | SARSA | Q-Learning |
|---|---|---|
| **Tipo** | On-policy | Off-policy |
| **Objetivo** | r + γ Q(s',a') | r + γ max Q(s',·) |
| **Aprende** | $Q^{\pi}$ (de la política ε-greedy) | Q* (de la política greedy) |
| **Estilo** | Cauteloso | Óptimo pero arriesgado al entrenar |

### FÓRMULAS

*Actualización*

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$$

*Política objetivo*

$$\pi(s) = \operatorname{argmax}_{a} Q(s,a)$$

### PASO A PASO

1. Inicializar Q(s,a) arbitrariamente.
2. En s elegir a con ε-greedy sobre Q.
3. Ejecutar a; observar r y s'.
4. Calcular el objetivo r + γ $\max_{a'}$ Q(s',a') (sin elegir a' realmente).
5. Actualizar Q(s,a) hacia ese objetivo.
6. s ← s'. Repetir hasta fin del episodio.

### EJEMPLO

***Cliff Walking.*** *Q-Learning converge a la ruta más corta junto al borde del acantilado (la óptima). Durante el entrenamiento, por la exploración ε, se cae más veces que SARSA; una vez entrenado y con ε = 0, su recorrido es mejor.*

### RELACIONADO · CLIC PARA ABRIR

Diferencia temporal — es TD aplicado a control
On-Policy vs Off-Policy — ejemplo off-policy
Epsilon-Greedy — explora con ε-greedy
Algoritmo SARSA — su contraparte on-policy
Algoritmo DQN (Deep Q-Network) — se extiende con redes neuronales
Función acción-valor — estima Q*

# Epsilon-Greedy (ε-greedy)

*La estrategia más simple para equilibrar explorar y explotar*

← Volver al mapa

Con probabilidad **1 − ε** el agente **explota**: elige la acción de mayor valor estimado; con probabilidad **ε explora**: elige una acción al azar. Garantiza que todas las acciones se sigan probando.

### IDEAS CLAVE

- **Dilema:** explotar maximiza la recompensa ahora; explorar puede descubrir algo mejor. Hace falta un equilibrio.
- **Decaimiento de ε:** se empieza con ε alto (≈ 1,0) para explorar mucho y se reduce (≈ 0,01–0,1) a medida que el agente aprende.
- **Dónde se usa:** SARSA, Q-Learning, DQN y MC control.
- **Alternativas:** softmax/Boltzmann (elige según probabilidades proporcionales al valor), UCB (optimismo ante la incertidumbre), inicialización optimista.

### FÓRMULAS

*Política ε-greedy*

$$\pi(a|s) = 1 - \varepsilon + \varepsilon/|A| \text{ si } a = \operatorname{argmax} Q(s,\cdot)$$

$$\pi(a|s) = \varepsilon/|A| \text{ en otro caso}$$

*Decaimiento exponencial*

$$\varepsilon_t = \max(\varepsilon_{min}, \varepsilon_0 \cdot d^t)$$

### EJEMPLO

***Elegir restaurante.*** *Con ε = 0,1: el 90 % de las veces vas a tu favorito (explotar) y el 10 % probás uno nuevo al azar (explorar). Si nunca probás, jamás sabrás si existe uno mejor.*

### RELACIONADO · CLIC PARA ABRIR

Algoritmo SARSA — lo usa para elegir a y a'
Algoritmo Q-Learning — lo usa como política de comportamiento
Algoritmo DQN (Deep Q-Network) — lo usa para recolectar experiencia
On-Policy vs Off-Policy — define la política de comportamiento
Componentes del aprendizaje por refuerzo — dilema exploración–explotación

# Algoritmo DQN (Deep Q-Network)

*Q-Learning con redes neuronales profundas*

← Volver al mapa

**DQN** (DeepMind, 2013/2015) reemplaza la Q-table por una **red neuronal profunda** Q(s,a;θ) que aproxima la función acción-valor. Permite aprender directamente de entradas de alta dimensión, como los **píxeles** de un videojuego Atari.

### IDEAS CLAVE

- **Experience Replay:** las transiciones (s,a,r,s',done) se guardan en un buffer y se entrena con *minibatches aleatorios*. Rompe la correlación entre muestras consecutivas y reutiliza datos.
- **Target Network:** una copia de la red con parámetros congelados $\theta^-$ calcula el objetivo y se actualiza cada C pasos. Evita perseguir un objetivo que se mueve en cada paso.
- **Off-policy y basado en valores;** acciones discretas; explora con ε-greedy.
- **Extensiones:** Double DQN (reduce la sobreestimación), Dueling DQN, Prioritized Replay, Rainbow.

### PASO A PASO

1. Observar s, elegir a con ε-greedy usando la red $Q(s,\cdot;\theta)$.
2. Ejecutar a, observar r, s'; guardar (s,a,r,s') en el buffer.
3. Muestrear un minibatch aleatorio del buffer.
4. Calcular el objetivo $y = r + \gamma \max Q(s',\cdot;\theta^-)$ con la red objetivo.
5. Minimizar $(y - Q(s,a;\theta))^2$ por descenso del gradiente.
6. Cada C pasos copiar $\theta^- \leftarrow \theta$.

### FÓRMULAS

*Función de pérdida*

$$L(\theta) = E_{(s,a,r,s')\sim D}\left[ \left( r + \gamma \max_{a'} Q(s',a';\theta^-) - Q(s,a;\theta) \right)^2 \right]$$

### EJEMPLO

***Atari (Breakout, Pong…).*** *La entrada son 4 fotogramas de 84×84 píxeles apilados; la red convolucional devuelve un Q para cada botón del joystick. DQN superó el nivel humano en muchos juegos con el mismo algoritmo y la misma arquitectura.*

### RELACIONADO · CLIC PARA ABRIR

Algoritmo Q-Learning — es Q-Learning con red neuronal
Función acción-valor — aproxima Q(s,a)
On-Policy vs Off-Policy — off-policy gracias al replay
Epsilon-Greedy — explora con ε-greedy
Aprendizaje basado en valores vs. basado en políticas — basado en valores

# Policy Gradient y algoritmo REINFORCE

*Optimizar la política directamente por ascenso del gradiente*

← Volver al mapa

Los métodos de **gradiente de la política** parametrizan la política $\pi_\theta(a|s)$ (típicamente una red neuronal) y ajustan $\theta$ para **aumentar el retorno esperado** $J(\theta)$. **REINFORCE** (Williams, 1992) es el algoritmo más simple: usa retornos Monte Carlo de episodios completos.

### IDEAS CLAVE

- **Intuición:** si una acción llevó a un buen retorno, aumentar su probabilidad; si llevó a uno malo, reducirla.
- **Teorema del gradiente de la política:** el gradiente se puede estimar sin conocer el modelo, solo con $\nabla \log \pi$ y retornos.
- **Ventajas:** maneja acciones continuas, aprende políticas estocásticas y su convergencia es suave.
- **Desventajas:** alta varianza, baja eficiencia de muestras y necesita episodios completos; converge a óptimos locales.
- **Baseline:** restar b(s) al retorno reduce la varianza sin introducir sesgo. Llevarlo al extremo da el actor-crítico (A2C).

### PASO A PASO

1. Inicializar los parámetros $\theta$ de la política.
2. Generar un episodio completo siguiendo $\pi_\theta$: $(S_0, A_0, R_1, \ldots, S_T)$.
3. Para cada paso t calcular el retorno $G_t$.
4. Actualizar $\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla_\theta \log \pi_\theta(A_t|S_t)$.
5. Repetir con un nuevo episodio.

### FÓRMULAS

*Objetivo*

$$J(\theta) = E_{\pi_\theta}[\, G_0 \,]$$

*Gradiente de la política*

$$\nabla_\theta J(\theta) = E_{\pi_\theta}[\, \nabla_\theta \log \pi_\theta(a|s) \cdot G_t \,]$$

*REINFORCE*

$$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla_\theta \log \pi_\theta(A_t|S_t)$$

*Con baseline*

$$\theta \leftarrow \theta + \alpha \cdot (\, G_t - b(S_t) \,) \cdot \nabla_\theta \log \pi_\theta(A_t|S_t)$$

### EJEMPLO

***CartPole.*** *Una red con softmax da la probabilidad de empujar el carro a la izquierda o derecha. Al terminar cada episodio, las acciones de los episodios largos (que mantuvieron el palo en equilibrio) ven aumentar su probabilidad.*

### RELACIONADO · CLIC PARA ABRIR

Aprendizaje basado en valores vs. basado en políticas — familia basada en políticas

Monte Carlo — usa retornos Monte Carlo

A2C (Advantage Actor-Critic) — se mejora agregando un crítico

On-Policy vs Off-Policy — es on-policy

# A2C — Advantage Actor-Critic

*Actor-crítico con función de ventaja, actualización síncrona*

← Volver al mapa

**A2C** combina dos redes (o dos cabezas de una misma red): el **actor** $\pi_\theta(a|s)$, que decide, y el **crítico** $V_w(s)$, que evalúa los estados y se aprende por TD. El crítico permite calcular la **ventaja**, que reemplaza al retorno en el gradiente y reduce mucho la varianza.

### IDEAS CLAVE

- **Ventaja A(s,a):** cuánto mejor fue la acción que el valor promedio del estado. Se estima con el error TD: $A \approx r + \gamma V(s') - V(s)$.
- **Actor:** sube la probabilidad de acciones con ventaja positiva y baja las de ventaja negativa.
- **Crítico:** se entrena para predecir bien V(s) minimizando el error TD cuadrático.
- **Síncrono:** varios entornos paralelos avanzan n pasos; se esperan todos, se promedian los gradientes y se hace **una** actualización.
- **Compromiso:** menos varianza que REINFORCE a cambio de algo de sesgo (por el bootstrapping del crítico). Es on-policy.
- **Pérdida total:** pérdida de política + $c_1$·pérdida del crítico − $c_2$·entropía (la entropía fomenta explorar).

### FÓRMULAS

*Ventaja (error TD)*

$$A(s,a) \approx r + \gamma\, V_w(s') - V_w(s)$$

*Actor*

$$\theta \leftarrow \theta + \alpha_\theta \cdot A \cdot \nabla_\theta \log \pi_\theta(a|s)$$

*Crítico*

$$w \leftarrow w - \alpha_w \nabla_w \left( r + \gamma\, V_w(s') - V_w(s) \right)^2$$

### EJEMPLO

***Actor y crítico.*** *El actor es un jugador; el crítico, su entrenador: no le dice qué hacer, sino «esa jugada salió mejor (o peor) de lo que esperaba». El jugador ajusta su estilo según esa retroalimentación.*

### RELACIONADO · CLIC PARA ABRIR

Policy Gradient y algoritmo REINFORCE — mejora REINFORCE con un crítico
A3C (Asynchronous Advantage Actor-Critic) — su versión asíncrona
Diferencia temporal — el crítico se aprende con TD
Función estado-valor — el crítico estima V(s)
Aprendizaje basado en valores vs. basado en políticas — método híbrido valor + política

# A3C — Asynchronous Advantage Actor-Critic

*Actor-crítico entrenado por muchos trabajadores en paralelo*

← Volver al mapa

**A3C** (Mnih et al., DeepMind, 2016) lanza varios **workers asíncronos**. Cada uno tiene una copia local de la red y su propio entorno, calcula gradientes con su experiencia y los aplica a una **red global compartida sin esperar** a los demás.

### IDEAS CLAVE

- **Decorrelación sin replay buffer:** como cada worker explora situaciones distintas, los datos son diversos sin necesidad de guardar experiencia.
- **Eficiente en CPU multinúcleo;** no requiere GPU. Entrenó bien juegos Atari en menos tiempo que DQN.
- **Ciclo de cada worker:** copiar parámetros globales → jugar n pasos → calcular gradientes con ventajas → enviarlos a la red global.
- **Es on-policy,** usa retornos de n pasos y suele agregar entropía para fomentar la exploración.
- **A2C vs A3C:** A2C es la versión síncrona; en la práctica, con GPU, A2C suele rendir igual o mejor y es más fácil de reproducir.

### A2C VS A3C

| | A2C | A3C |
|---|---|---|
| **Actualización** | Síncrona: espera a todos los workers | Asíncrona: cada worker actualiza cuando termina |
| **Parámetros** | Una sola red, un solo update por lote | Red global + copias locales |
| **Hardware ideal** | GPU (lotes grandes) | CPU multinúcleo |
| **Reproducibilidad** | Alta (determinista) | Menor (orden de actualización variable) |
| **Política** | On-policy | On-policy |

### FÓRMULAS

*Retorno de n pasos*

$$R_t = \Sigma_{i=0}^{n-1} \gamma^i r_{t+i} + \gamma^n V(s_{t+n})$$

*Ventaja*

$$A_t = R_t - V(s_t)$$

*Gradiente por worker*

$$d\theta \leftarrow d\theta + \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A_t$$

### EJEMPLO

***Equipo de exploradores.*** *Varios exploradores recorren partes distintas de un territorio y cada uno actualiza un mapa común apenas descubre algo útil, sin esperar a los otros. El mapa mejora más rápido que con un solo explorador.*

### RELACIONADO · CLIC PARA ABRIR

A2C (Advantage Actor-Critic) — su versión síncrona
Policy Gradient y algoritmo REINFORCE — heredan el gradiente de la política
On-Policy vs Off-Policy — es on-policy
Algoritmo DQN (Deep Q-Network) — otra vía de estabilizar el aprendizaje profundo

# Cuadro comparativo de algoritmos

*Resumen para integrar los contenidos*

← Volver al mapa

| Algoritmo | Familia | Política | Qué aprende | Regla de actualización (resumen) | Acciones | ¿Modelo? |
|---|---|---|---|---|---|---|
| **Prog. dinámica** | Valor (planificación) | — | $V^*$, $Q^*$, $\pi^*$ | $V(s) \leftarrow \max_a \Sigma\, P(s' \mid s,a)[\, r + \gamma\, V(s') \,]$ | Discretas | Sí |
| **Monte Carlo** | Valor | On / off (con IS) | $V^\pi$, $Q^\pi$ | $V(s) \leftarrow V(s) + \alpha\, [\, G_t - V(s) \,]$ | Discretas | No |
| **TD(0)** | Valor | On / off | $V^\pi$ | $V(s) \leftarrow V(s) + \alpha\, [\, r + \gamma\, V(s') - V(s) \,]$ | Discretas | No |
| **SARSA** | Valor | On-policy | $Q^\pi \to Q^*$ | $Q(s,a) \leftarrow Q(s,a) + \alpha\, [\, r + \gamma\, Q(s',a') - Q(s,a) \,]$ | Discretas | No |
| **Q-Learning** | Valor | Off-policy | $Q^*$ | $Q(s,a) \leftarrow Q(s,a) + \alpha\, [\, r + \gamma \max Q(s',\cdot) - Q(s,a) \,]$ | Discretas | No |
| **DQN** | Valor (profundo) | Off-policy | $Q(s,a;\theta)$ con red | $\min\, (\, r + \gamma \max Q(s',\cdot;\theta^-) - Q(s,a;\theta) \,)^2$ con replay y red objetivo | Discretas | No |
| **REINFORCE** | Política | On-policy | $\pi_\theta$ | $\theta \leftarrow \theta + \alpha\, G_t\, \nabla \log \pi_\theta(a \mid s)$ | Disc. y cont. | No |
| **A2C** | Híbrido actor-crítico | On-policy | $\pi_\theta$ y $V_w$ | $\theta \leftarrow \theta + \alpha\, A\, \nabla \log \pi_\theta$; $A = r + \gamma\, V(s') - V(s)$; síncrono | Disc. y cont. | No |
| **A3C** | Híbrido actor-crítico | On-policy | $\pi_\theta$ y $V_w$ | Igual que A2C pero con workers asíncronos sobre una red global | Disc. y cont. | No |

*IS = importance sampling.  Clic en el nombre de un algoritmo para abrir su explicación.*

### GUÍA RÁPIDA PARA ELEGIR UN ALGORITMO

| | |
|---|---|
| **¿Tengo el modelo del entorno?** | Sí → programación dinámica. No → Monte Carlo, TD y derivados. |
| **¿Episodios cortos, o tareas continuas?** | Episodios: MC o TD. Continuas: TD. |
| **¿Acciones discretas o continuas?** | Discretas: SARSA, Q-Learning, DQN. Continuas: métodos de política (REINFORCE, A2C/A3C). |
| **¿Estados enormes (imágenes)?** | Aproximar con redes: DQN (valor) o A2C/A3C (política + valor). |
| **¿Quiero reutilizar experiencia?** | Off-policy: Q-Learning, DQN. Si no, on-policy: SARSA, REINFORCE, A2C/A3C. |